Robotic Tufting — Weekly Plan

Step 2: Camera, Frame Detection & Image Mapping

Week of March 27 – March 31, 2026

Your responsibility in one sentence: Write a Python module (`frame_scanner.py`) that uses the ZED 2i camera to scan the tufting frame, maps any input image onto the canvas, and exports a list of physical 3D coordinates — one per tuft point — ready for the path planner.

What You Need Before You Start

Physical Items

- **ZED 2i camera** — you already have this.
- **Printed ChArUco board** — this is the only physical item you are missing. Ask a teammate to print it today (Friday). It must be printed on rigid, flat material such as foam board or thick cardboard — not plain paper, which can curl. After printing, measure the squares with a ruler and confirm the size matches what is specified in the code. If a square prints as 28 mm instead of 30 mm, update the value in the code accordingly.
- **Tufting frame, assembled and in position** — you need the bare frame placed where it will actually be used during the project. No yarn, no gun needed yet.

What You Do NOT Need Yet

- The hand-eye calibration matrix (that is another teammate's task — you can add it later as a single line of code).
- The robot to be running or connected.
- The tufting gun.

Most urgent action: Get the ChArUco board printed today. Everything else depends on it.

Day-by-Day Plan

Friday — Software Setup & Skeleton (Today)

Goal: Install everything, confirm the camera works, write the full code skeleton.

1. **Install all required libraries.** Open a terminal and run:

```
pip install opencv-contrib-python numpy Pillow matplotlib
```

Make sure you install `opencv-contrib-python` and not plain `opencv-python` — the difference matters because the ChArUco detection tools only exist in the `contrib` version. Also install the ZED SDK from the Stereolabs website if not already done.

2. **Test the ZED 2i camera.** Connect the camera, open the ZED Explorer application (comes with the SDK), and confirm that you can see a live depth image. Then write a short Python script that simply opens the camera and prints the intrinsic values (focal length, optical center). If those numbers print correctly, your camera connection is working.

3. **Test ChArUco detection offline, without the real board.** Download any ChArUco board image from the internet and run your detection code against it as a static image file. This lets you confirm that detection logic works before the physical board is printed. You should see green dots drawn over the board corners in the output image.

4. **Write the full module skeleton.** Create `frame_scanner.py` with the following functions, each doing its job independently:

   - `init_camera()` — opens the ZED 2i and retrieves intrinsic parameters
   - `detect_frame()` — detects the ChArUco board and estimates its 3D pose
   - `build_grid()` — creates an evenly spaced grid of tuft points across the canvas
   - `map_image()` — resizes the input image and assigns a colour to each grid point
   - `export_points()` — saves the final list of coordinates to a JSON file

   You do not need these to be fully working today — just written, structured, and runnable without crashing.

Saturday — Live Detection & Frame Scanning

Goal: Board is printed, frame is set up — run detection live and confirm coordinates are correct.

1. **Place the ChArUco board on the canvas.** Lay it flat against the top-left corner of the tufting frame, flush with the canvas surface. Tape or pin it down so it cannot move or curl. The board's bottom-left corner becomes the origin point (0, 0) of your coordinate system.

2. **Measure the canvas offsets.** With a ruler, measure:

   - How wide the canvas is in millimetres (total width)
   - How tall the canvas is in millimetres (total height)
   - How far the canvas extends beyond the board's right edge
   - How far the canvas extends below the board's bottom edge

   Write these numbers down — they go directly into your code as constants.

3. **Run live detection.** Point the ZED 2i at the board, run `detect_frame()`, and draw the detected corners back onto the camera image. You should see coloured dots sitting

precisely on each inner corner of the board. If they are in the right place, your pose estimation is working.

4. **Project the canvas corners onto the image.** Using the board pose and your physical measurements, compute where the four canvas corners are in camera space, then project them back to 2D pixel positions and draw a rectangle on the live image. Confirm visually that the rectangle matches the actual frame edges. This is your most important verification step.

5. **Retrieve real depth values.** Switch on the ZED's point cloud and sample the depth at several points across the canvas. Confirm the values are finite numbers (not NaN) and physically plausible — for example, if the camera is 60 cm above the canvas, Z values should be around 600 mm.

Sunday — Image Mapping & End-to-End Test

Goal: Run the complete pipeline from image input to exported coordinate file.

1. **Prepare test images.** Have at least three test images ready: a simple smiley face, a filled circle, and a short letter or number. Keep them as PNG files with a plain white background — your code will treat white as empty space and skip those pixels.

2. **Run `map_image()`.** Load one of your test images, resize it to match the grid dimensions of the canvas (for example, if your canvas is 500 mm wide and you tuft every 5 mm, the image resizes to 100 pixels wide), and assign a colour to each grid point based on its corresponding pixel.

3. **Filter background pixels.** Any pixel brighter than a threshold (for example, all three RGB channels above 240) is treated as background and excluded from the output. Only coloured or dark pixels become actual tuft points.

4. **Export and visualise.** Run `export_points()` to save `tuft_points.json`. Then write a quick Matplotlib script that reads this file and plots all the points as a scatter chart — X on one axis, Y on the other, coloured by their RGB value. If the scatter plot looks like your input image, the pipeline is working end to end.

5. **Test with multiple images.** Swap in your other test images and re-run the pipeline each time. Confirm that different images produce different coordinate sets and that the output always looks like the input when plotted.

Monday — Presentation

Goal: Live demonstration of the full Step 2 pipeline.

1. Place the ChArUco board on the frame, run the scan, show the live camera feed with the detected board corners and projected canvas rectangle overlaid.

2. Load a test image (a smile or a logo), run the full pipeline, show the exported JSON file and the Matplotlib scatter visualisation.

3. Briefly explain what each step does and what goes to the path planner next.

Key Risks to Watch For

- **Board printed at wrong scale.** The moment the board comes off the printer, measure a square with a ruler. If it does not match the value in your code, update the code immediately. This takes 30 seconds and prevents hours of debugging.
- **Board not flat.** Any curl or tilt in the board introduces error into every 3D coordinate. Press it firmly against the canvas and tape all four edges.
- **ZED returning NaN depth values.** This happens at sharp angles or near the edges of the camera's view. Always filter out NaN values before using any depth reading. Move the camera closer or adjust its angle if too many points are invalid.
- **Detection failing in low light.** ChArUco detection requires reasonable lighting. If detection fails, add a lamp. Do not rely on ambient room light alone.

Week at a Glance

Day	What to do	What you need
Friday	Install libraries, test ZED, write code skeleton, test ChArUco detection on static image	Laptop, ZED camera
Saturday	Print board, set up frame, run live detection, verify canvas projection	ChArUco board, tufting frame
Sunday	Run full pipeline with test images, export JSON, visualise with Matplotlib	Board + frame in position
Monday	Present live scan, detection overlay, and image-to-coordinate mapping	Full setup

Definition of success for Monday: You can place the board on the frame, run one command, and show a scatter plot that looks like the input image — with every dot having a real physical coordinate in millimetres.